

Classes and Objects

Phạm Thị Kim Ngôn

`ngon.phamthikim@hoasen.edu.vn`

Objectives

1. Programming Paradigms
2. OOP basic concepts
3. How to identify classes
4. Hints for class design
5. How to declare/use a class
6. Common modifiers (a way to hide some members in a class)

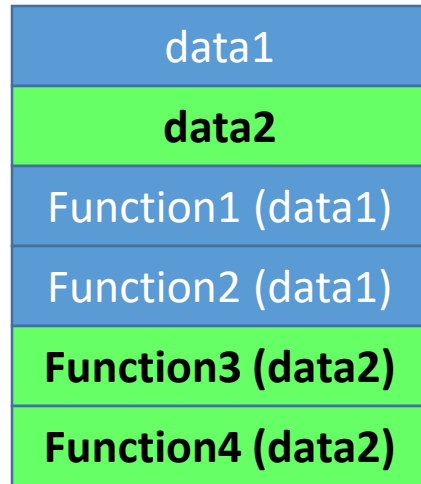
Programming Paradigms

- High-level programming languages

Paradigm	Description
Procedural-oriented (imperative) paradigm (POP)	Program= data + algorithms. Each algorithm is implemented as a function (group of statements) and data are it's parameters (C-language)
Object-oriented paradigm (OOP)	Programs = actions of some objects. Object = data + behaviors. Each behavior is implemented as a method (C++, Java, C#,...)
Functional paradigm	Domain-specific languages. Basic functions were implemented. Programs = a set of functions (F#)
Declarative/Logic paradigm	Program = declarations + inference rules (Prolog, CLISP, ...)

Programming Paradigms: POP vs. OOP

Procedure-Oriented Program



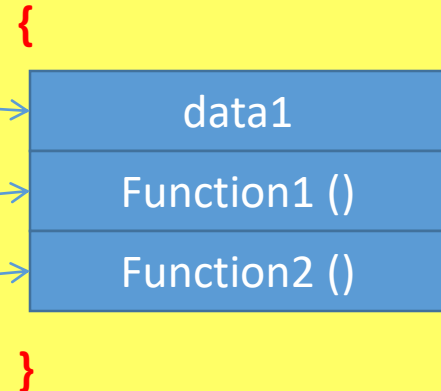
Object = Data + Methods

Basic Concepts

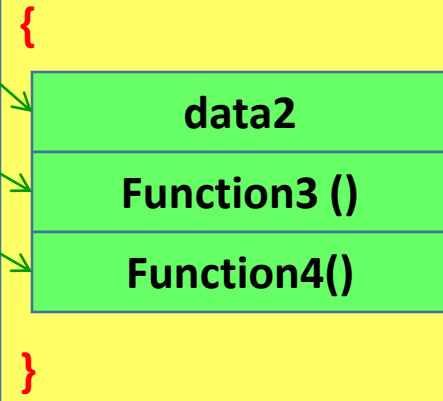
- Encapsulation
- Inheritance
- Polymorphism

Particular
methods:
Constructors

Class A



Class B



Common
methods
for
accessing a
data field:

Type getField()
void setField (Type newValue)

OOP Concepts

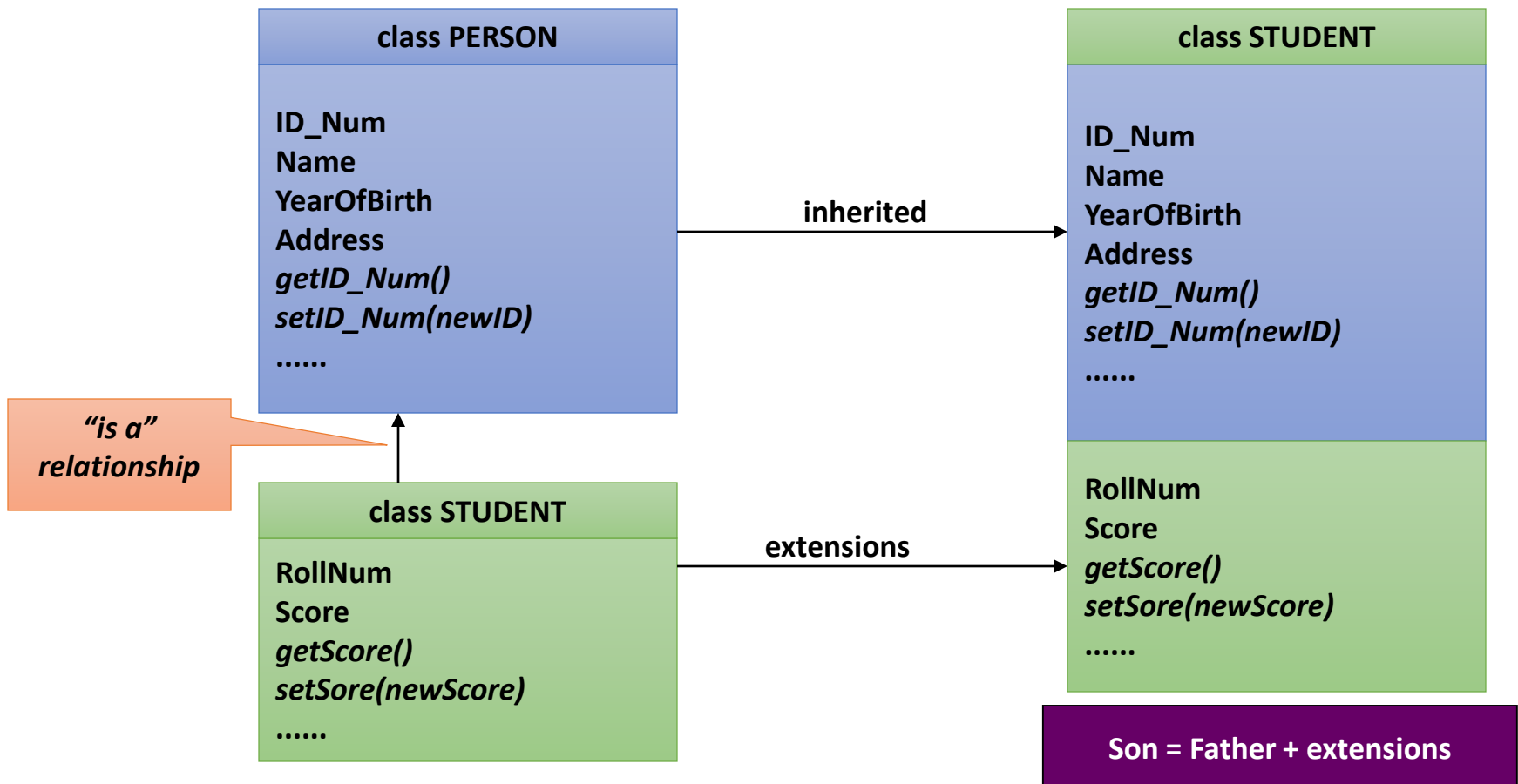
- Encapsulation
- Inheritance
- Polymorphism

Encapsulation

- Aggregation of data and behavior.
 - Class = Data (fields/properties) + Methods
 - Data of a class should be hidden from the outside.
 - All behaviors should be accessed only via methods.
 - A method should have a boundary condition: Parameters must be checked (use if statement) in order to assure that data of an object are always valid.
 - **Constructor**: A special method its code will execute when an object of this class is initialized.

Inheritance

- Ability allows a class having members of an existed class
- Re-used code, save time

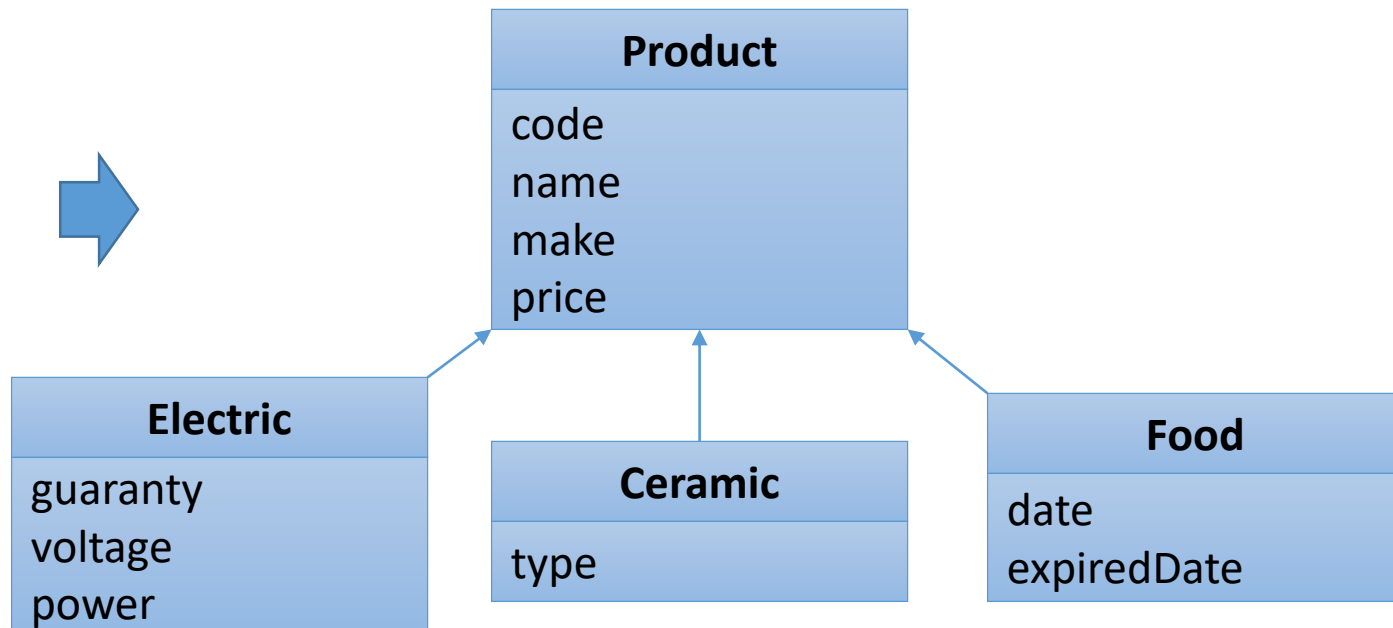


- How to detect father class?

Inheritance

Finding the intersection of concerned classes.

- Electric Products < **code, name, make, price**, guaranty, voltage, power>
- Ceramic Products < **code, name, make, price**, type >
- Food Products < **code, name, make, price** , date, expiredDate >



Polymorphism

- Ability allows many versions of a method based on overloading and overriding methods techniques.
 - **Overloading:** A class can have some methods which have the same name but their parameter types are different.
 - **Overriding:** A method in father class can be overridden in it's derived classes (body of a method can be replaced in derived classes).

How to Identity a Class

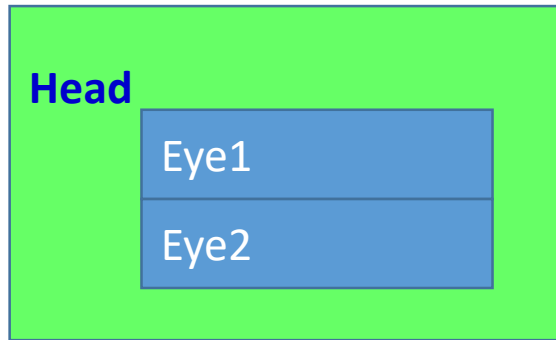
- Main noun: Class
- Nouns as modifiers of main noun: Fields
- Verbs related to main noun: Methods

Details of a **student** include **code**,
name, **year of birth**, **address**.

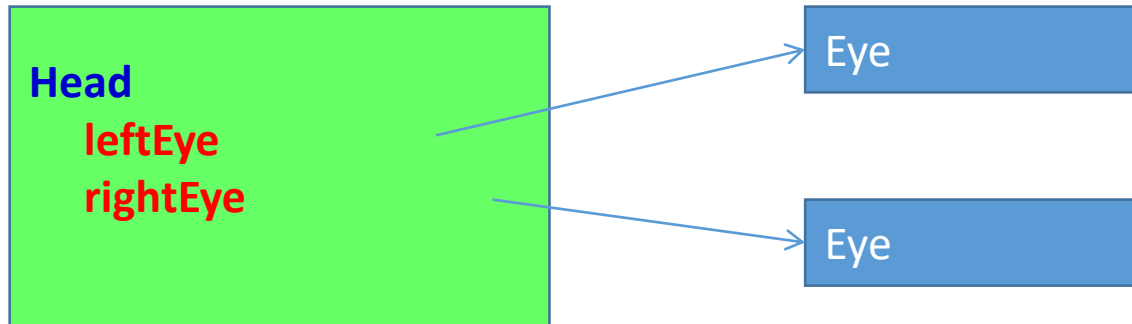
Write a Java program that will allow
input a student, **output** his/her.

```
class Student {  
    String code;  
    String name;  
    int bYear;  
    String address;  
    void input() {  
        <code>  
    }  
    void output() {  
        <code>  
    }  
}
```

Hints for class design

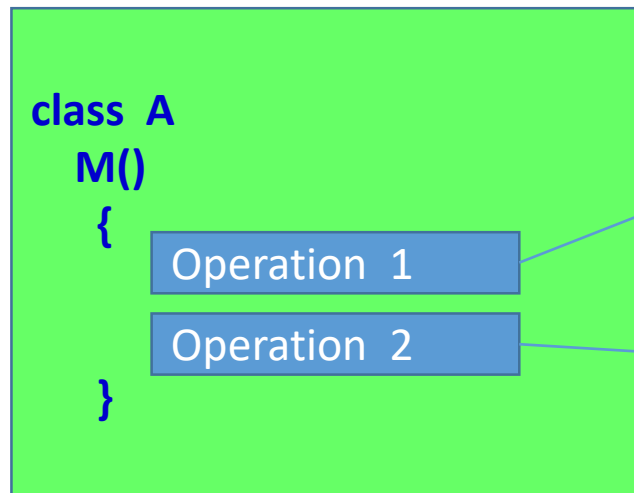


**High coupling
(Bad design)**

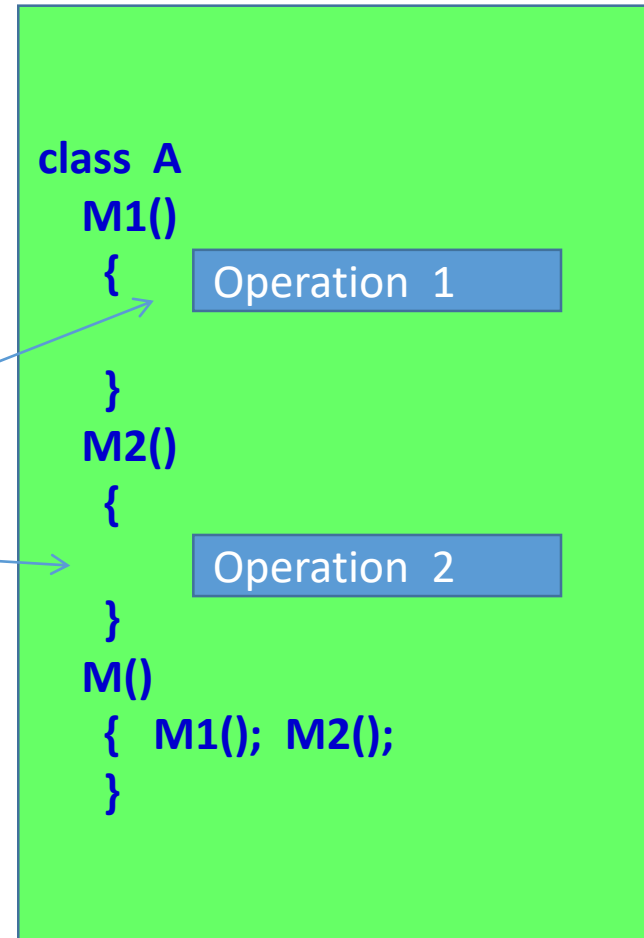


**Low coupling
(Good design)**

Hints for class design



Low cohesion (bad design)



High cohesion (good design)

Declaring/Using a Java Class

```
[public] class ClassName [extends FatherClass] {
```

```
    [modifier] Type field1 [= value];
```

```
    [modifier] Type field2 [= value];
```

```
    // constructor
```

```
    [modifier] ClassName (Type var1,...) {
```

```
        <code>
```

```
    }
```

```
    [modifier] Type methodName (Type var1,...) {
```

```
        <code>
```

```
    }
```

```
    .....  
}
```

Có thể định nghĩa nhiều hàm khởi tạo với các tham số khác nhau

How many constructors should be implemented? → Number of needed ways to initialize an object.

What should we will write in constructor's body? → They usually are codes for initializing values to descriptive variables

Defining Constructors

- Constructors that are invoked to create objects from the class blueprint. *Tự động gọi thực hiện khi tạo đối tượng*
- Constructor declarations look like method declarations—except that they use the name of the class and have no return type. *Tên hàm giống tên class và không có loại trả về*
- The compiler automatically provides a no-argument, **default constructor for any class without constructors.**

Nếu không định nghĩa hàm khởi tạo thì compiler sẽ tự cung cấp hàm khởi tạo mặc định (không có tham số)

Nếu class đã định nghĩa hàm khởi tạo thì compiler sẽ không tự động tạo và gọi hàm khởi tạo mặc định

Defining Methods

```
[modifier] ReturnType methodName (params) {  
    <code>  
}
```

- Signature: data help identifying something
- Method Signature:

name + order of parameter types

Passing Arguments a Constructor/Method

- Java uses the mechanism passing by value. Arguments can be:
 - Primitive Data Type Arguments
 - Reference Data Type Arguments (objects)

Creating Objects

- Class provides the blueprint for objects; you create an object from a class.

```
Point p = new Point(23, 94);
```

- Statement has three parts:
 - **Declaration**: are all variable declarations that associate a variable name with an object type.
 - **Instantiation**: The new keyword is a Java operator that creates the object (memory is allocated).
 - **Initialization**: The new operator is followed by a call to a constructor, which initializes the new object (values are assigned to fields).

Type of Constructors

Create/Use an object of a class

- Default constructor: Constructor with no parameter.
- Parametric constructor: Constructor with at least one parameter.
- Create an object

```
ClassName obj1=new ClassName();
```

```
ClassName obj2=new ClassName(params);
```

- Accessing a field of the object: `object.field`
- Calling a method of an object: `object.method(params)`

Demo: If we do not implement any constructor, compiler will insert to the class a default constructor

```
package point1;  
public class IntPoint1 {  
    int x;  
    int y;  
    // If no constructor is implemented, the compiler will insert  
    // automatically a default constructor to the class  
    public void output(){  
        System.out.println ("[" + x + "," + y + "]");  
    }  
}
```

System constructor will clear all bits
in allocated memory

Order for
initializing an
object

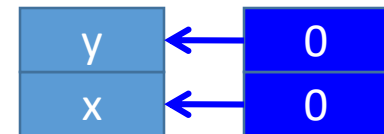
Setup
values

```
1 package point1;  
2 public class IntPoint1_Use {  
3     public static void main (String[] args){  
4         // Create a point using default constructor  
5         IntPoint1 p = new IntPoint1();  
6         p.output();  
7     }  
8 }
```

Output - FirstPrj (run) x

```
run:  
[0,0]  
BUILD SUCCESSFUL (total time: 0 seconds)
```

An object variable is a reference



Demo: If we implement a constructor, compiler does not insert default constructor

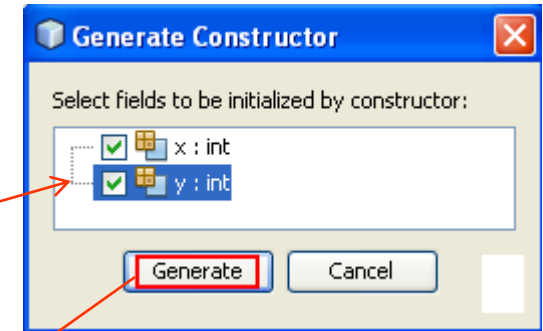
Insert constructor

```
package point1;  
public class IntPoint2 {  
    int x;  
    int y;  
}
```

Navigate
Show Javadoc Alt+F1
Find Usages Alt+F7
Call Hierarchy
Insert Code... Alt+Insert

```
package point1;  
public class IntPoint2 {  
    int x;  
    int y;  
}
```

Generate
Constructor...
Logger...
Getter...
Setter...
Getter and Setter...



```
package point1;  
public class IntPoint2 {  
    int x;  
    int y;  
  
    public IntPoint2(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

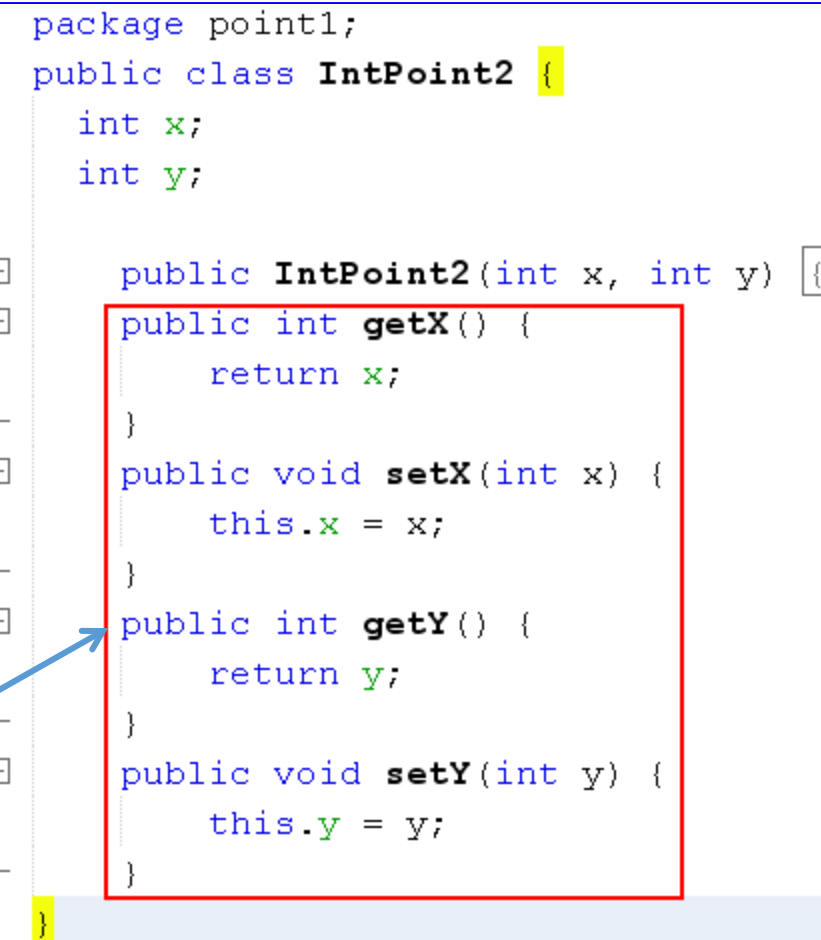
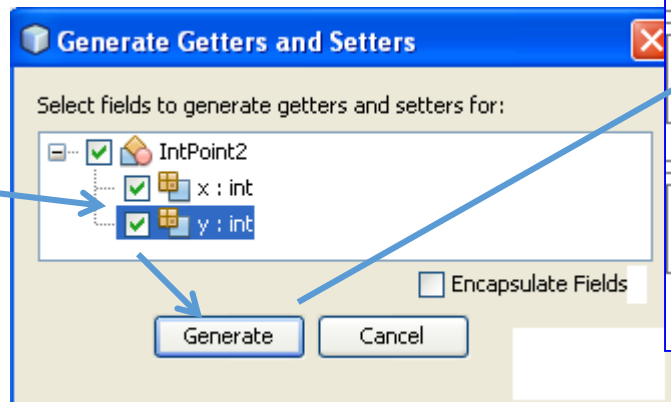
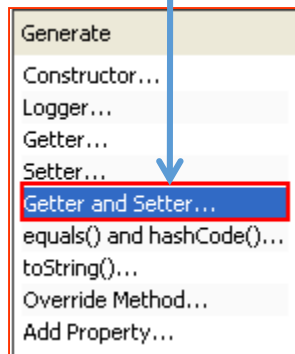
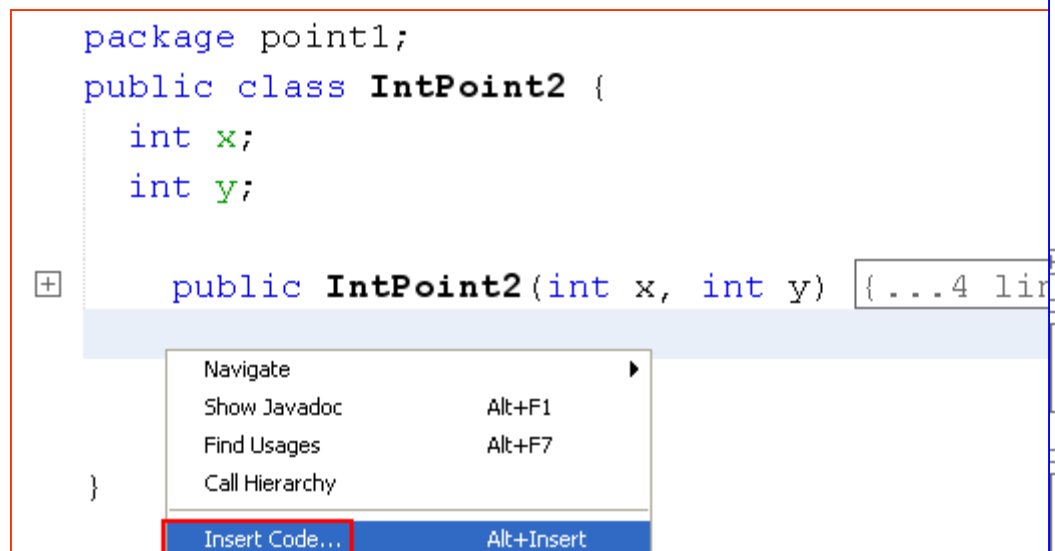
Parameter names are the same as those in declared data filed. So, the keyword **this** will help distinguish field name and parameter name.

this.x means that x of this object

Demo: If we implement a constructor, compiler does not insert default constructor

Accessing each data field is usually supported by :
A getter for reading value of this field
A setter for modifying this field

Insert getter/setter



Demo: If we implement a constructor, compiler does not insert system constructor

```
package point1;
public class IntPoint2 {
    int x;
    int y;

    public IntPoint2(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { ...3 lines }
    public void setX(int x) { ...3 li
    public int getY() { ...3 lines }
    public void setY(int y) { ...3 li
}
```

```
1 package point1;
2 public class IntPoint2_Use {
3     public static void main (String[] args){
4         // Create a point using default constructor
5         // Error:Constructor InPoint2 in class IntPoint2 can
6         // not be applied to given type;required: int, int
7         IntPoint2 p = new IntPoint2();
8     }
9 }
```

Explain the result of the following program

```
package point1;

public class IntPoint2 {
    int x=7;
    int y=3;
    public IntPoint2(){
        output();
        x=100;
        y=1000;
        output();
    }

    public IntPoint2(int x, int y) {
        output();
        this.x = x;
        this.y = y;
        output();
    }

    public void output(){
        String S= "[" + x + "," + y + "]";
        System.out.println(S);
    }
}
```

```
package point1;

public class IntPoint2_Use {
    public static void main (String[] args){
        System.out.println("Use default constructor:");
        IntPoint2 p1= new IntPoint2();
        System.out.println("Use parametric constructor:");
        IntPoint2 p2 = new IntPoint2(-7,90);
    }
}
```

Output - FirstPrj (run) x

```
run:
Use default constructor:
[7,3]
[100,1000]
Use parametric constructor:
[7,3]
[-7,90]
BUILD SUCCESSFUL (total time: 0 seconds)
```

Common Modifiers

- Modifiers (OOP) are **keywords** that give the compiler information about the methods, data, classes.
- Java supports some modifiers in which some of them are common and they are called as access modifiers (public, protected, default, private).
- Common modifiers will impose level of accessing on
 - class (where it can be used?)
 - methods (whether they can be called or not)
 - fields (whether they may be read/written or not)

Outside of a Class

```
package point1;  
public class IntPoint2 {
```

```
    int x=7;  
    int y=3;  
    public IntPoint2(){  
        output();  
        x=100;  
        y=1000;  
        output();  
    }
```

```
    public IntPoint2(int x, int y) {  
        output();  
        this.x = x;  
        this.y = y;  
        output();  
    }
```

```
    public void output(){  
        String S= "[" + x + "," + y + "];"  
        System.out.println(S);  
    }
```

```
package point1;
```

```
public class IntPoint2_Use {
```

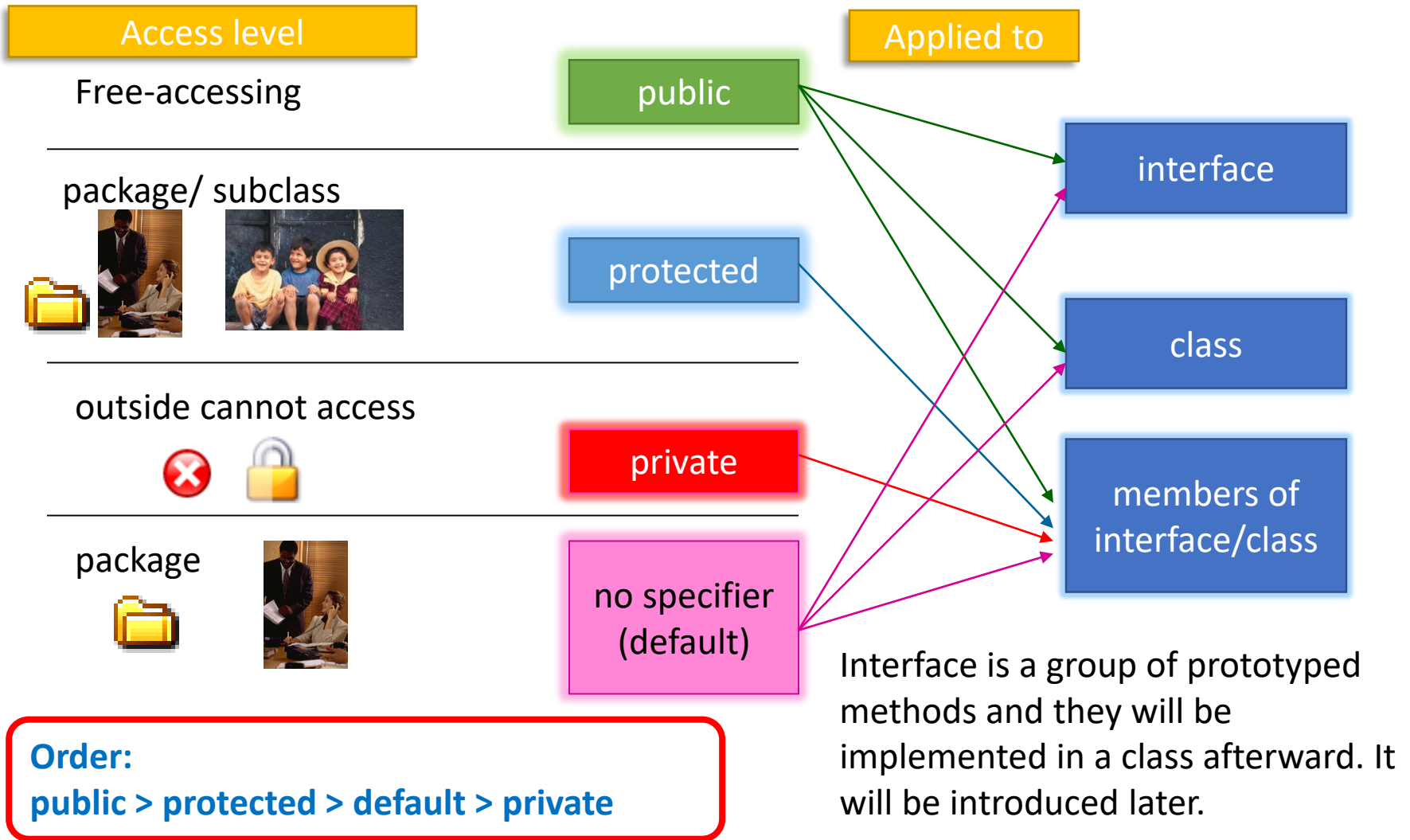
```
    public static void main (String[] args){  
        System.out.println("Use default constructor:");  
        IntPoint2 p1= new IntPoint2();  
        System.out.println("Use parametric constructor:");  
        IntPoint2 p2 = new IntPoint2(-7,90);  
    }
```

Inside of the class
IntPoint2

Inside of the class
IntPoint2_Use and it is
outside of the class
IntPoint2

Outside of the class A is another class
where the class A is accessed (used)

Common Modifiers



Access Level

Modifier	Class	Same Package	Subclass- Outside package	World
private	Y	N	N	N
No (default)	Y	Y	N	N
protected	Y	Y	Y	N
public	Y	Y	Y	Y

Common Modifiers

The screenshot displays an IDE with three Java files open:

- Rectangle.java**:

```
1 package rectPkg;  
2 public class Rectangle {  
3     protected int length;  
4     public int width;  
5     public void setSize (int l, int w)  
6     { length = l>0? l: 0;  
7       width = w>0? w: 0;  
8     }  
9 }
```
- Box.java**:

```
1 package boxPkg;  
2 import rectPkg.Rectangle;  
3 public class Box extends Rectangle {  
4     int height;  
5     protected int price;  
6     private int weight;  
7     void setSize(int l, int w, int h)  
8     { super.setSize(l,w);  
9       height = h>0? h: 0;  
10    }  
11    int volume ()  
12    { return length*width*height;  
13    }  
14 }
```
- Demo_1.java**:

```
1 package boxPkg;  
2 import rectPkg.Rectangle;  
3 public class Demo_1 {  
4     public static void main (String[] args)  
5     { Box b = new Box();  
6       b.setSize(1,2,3);  
7       b.height=10;  
8       b.price= 7;  
9       b.weight=9;  
10      System.out.println("Volumn of the box:" + b.volume());  
11      Rectangle r= new Rectangle();  
12      r.setSize(3,5);  
13      r.width=3;  
14      r.length=6;  
15    }  
16 }
```

Red arrows point from the `super.setSize(l,w)` call in `Box.java` and the `b.setSize(1,2,3)` call in `Demo_1.java` to the `setSize` method in `Rectangle.java`. Blue arrows point from the `super` keyword in `Box.java` and the `new Rectangle()` call in `Demo_1.java` to the `Rectangle` class in `Rectangle.java`.

super: Keyword for calling a member declared in the father class.
If contructor of sub-class calls a constructor of it's father using super, it must be the first statement in the sub-class constructor.

Demo: Overloading Method

```
/* Overloading methods Demo. */
public class Box {
    int length=0;
    int width=0;
    int depth=0;

    // Overloading constructors
    public Box(){
    }
    public Box(int l){
        length = l>0? l: 0; // safe state
    }
    public Box(int l, int w){
        length = l>0? l: 0; // safe state
        width = w>0? w: 0;
    }
    public Box(int l, int w, int d){
        length = l>0? l: 0; // safe state
        width = w>0? w: 0;
        depth = d>0? d: 0;
    }
}
```

```
// Overloading methods
public void setEdge (int l,int w){
    length = l>0? l: 0; // safe state
    width = w>0? w: 0;
}
public void setEdge (int l,int w,int d){
    length = l>0? l: 0; // safe state
    width = w>0? w: 0;
    depth = d>0? d: 0;
}

public void output(){
    String S= "[" + length + "," + width
              + "," + depth + "]";
    System.out.println(S);
}
```

```
/* Use the class Box */
public class BoxUse {
    public static void main(String[] args){
        Box b= new Box();
        b.output();
        b.setEdge(7,3);
        b.output();
        b.setEdge(90,100,75);
        b.output();
    }
}
```

Output - FirstPrj (run) x



run:



[0,0,0]



[7,3,0]

[90,100,75]

Access Modifier Overridden

The screenshot shows an IDE with a project named 'Chapter02'. Under 'Source Packages', there is 'boxPkg' containing 'Box.java' and 'Demo_1.java', and 'overridenDemo' containing 'ClassA.java' and 'ClassB.java'. The 'overridenDemo' package is highlighted. The 'ClassA.java' file shows the following code:

```
1 package overridenDemo;
2 public class ClassA {
3     public void M1() { }
4     protected void M2() { }
5     void M3() { }
6     private void M4() { }
7 }
8
```

The 'ClassB.java' file shows the following code:

```
1 package overridenDemo;
2 public class ClassB extends ClassA {
3     protected void M1() { }
4     void M2() { }
5     private void M3() { }
6     void M4() { }
7 }
```

Legal

private

default

protected

public

Illegal

public

protected

default

private

The sub-class must be more opened than it's father

Modifier *final*

- Final data is a constant.
- Final method: a method can not be overridden.

```
2 public class OtherModifierDemo{
3     final public int MAXN = 5;
4     public static void main(String[] args)
5     { OtherModifierDemo obj= new OtherModifierDemo();
6       obj.MAXN = 1000;
7       final int N=7;
8       N=10;
9     }
10 }
```

```
1 class A{
2     final void M() { System.out.println("MA"); }
3 }
4 class B extends A {
5     void M() { System.out.println("MB"); }
6 }
7 public class FinalMethodDemo {
8 }
9 }
```

Modifier *static*

Class variable/ Object variable

- Object variable: Variable of each object
- Class Variable: A variable is shared in all objects of class. It is stored separately. It is declared with the modifier *static*
- Class variable is stored separately. So, it can be accessed as:

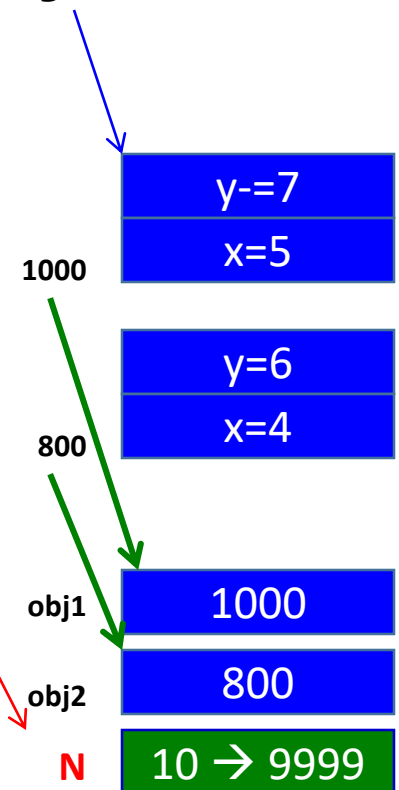
object.staticVar

ClassName.staticVar

Modifier *static*: Class variable/ Object variable

```
public class StaticVarDemo {  
    static int N=10; // class variable  
    int x, y; // object variable  
    public StaticVarDemo(int xx, int yy){  
        x= xx; y=yy;  
    }  
    public void setN( int nn){  
        N= nn;  
    }  
    public void output(){  
        System.out.println(N + "," + x + "," + y);  
    }  
}
```

```
public class StaticVarDemoUse {  
    public static void main(String args[]){  
        StaticVarDemo obj1= new StaticVarDemo(5,7);  
        StaticVarDemo obj2= new StaticVarDemo(4,6);  
        obj1.output();  
        obj2.output();  
        obj1.setN(9999);  
        obj1.output();  
        obj2.output();  
        System.out.println(StaticVarDemo.N);  
    }  
}
```



Output - FirstPrj (run) x

```
run:  
10,5,7  
10,4,6  
9999,5,7  
9999,4,6  
9999
```

Modifier *static*: Static method

- It is called as class method/global method and it is called although no object of this class is created.
- Entry point of Java program is a static method
- Syntax for calling it: **ClassName.staticMethod(args)**
- ***Static methods***:
 - can access class variables and class methods directly **only**.
 - ***cannot*** access instance variables or instance methods directly—they must use an object reference.
 - **cannot** use the **this** keyword as there is no instance for this to refer to.

Modifier *static*: Static method

```
11 class MyClass {
12     static int staticVar = 10; // Static variable
13     int instanceVar = 20;      // Instance variable
14
15     static void staticMethod() {
16         // Can access static variable
17         System.out.println("Static variable: " + staticVar);
18         System.out.println("Instance variable: " + instanceVar);
19         // Error: Cannot access instance variable
20
21         MyClass obj = new MyClass(); // Create instance
22
23         // Access instance variable via object reference
24         System.out.println("Instance variable: " + obj.instanceVar);
25
26     }
27
28     void instanceMethod() {
29         System.out.println("Instance method called");
30     }
31 }
```

Modifier *static*: **What should be static in a class?**

Constants:

- The static modifier, in combination with the final modifier, is also used to define constants. The final modifier indicates that the value of this field cannot change.

static final double PI = 3.141592653589793;

Demo: Methods with Arbitrary Number of Arguments

A group is treated as an array
group.length → number of elements
group[i]: The element at the position i

```
1 public class ArbitraryDemo {
2     public double sum(double... group){
3         double S=0;
4         for (double x: group) S+=x;
5         return S;
6     }
7     public String concat(String... group){
8         String S="";
9         for (String x: group) S+=x + " ";
10        return S;
11    }
12    public static void main(String[] args){
13        ArbitraryDemo obj= new ArbitraryDemo();
14        double total= obj.sum(5.4, 3.2, 9.08, 4);
15        System.out.println(total);
16        String line = obj.concat("I", "love", "you", "!");
17        System.out.println(line);
18    }
19 }
```

Output - FirstPrj (run) x



run:
21.68
I love you !



Q&A